

City University of Hong Kong 2025-2026
Semester A CS3343 Software Engineering
Practice

Test Report
Group 19
Classic UNO Card Game

Conducted By:

Woo King Hei 58533231

Wong Ka Chung 58533673

Chan Pak Yuen 58535918

Chan Wing Ki 58111083

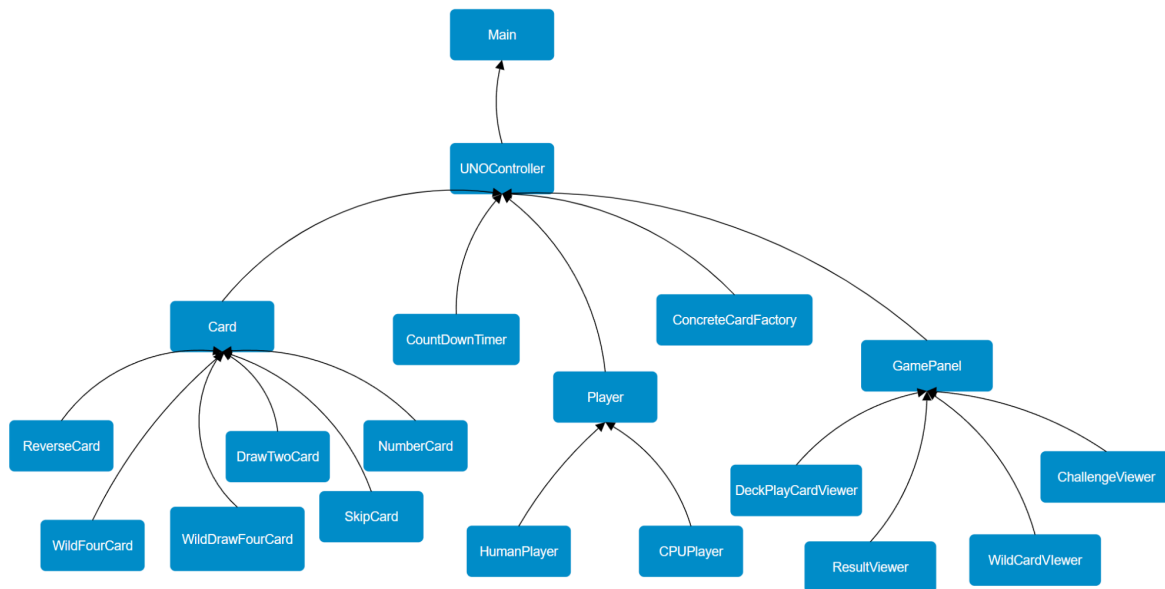
Ho Sung Hin 58010837

Wong Hei Leong 58537010



1. Hierarchy Diagram.....	3
2. Testing Approach.....	3
3. Testing Process.....	3
4. Coverage Analysis.....	6
5. Code Refactoring.....	8
5.1 Renaming Methods/Variables.....	8
5.2 Pull Up Field.....	9
5.3 Decompose Conditional.....	10
5.4 Extract Method.....	11
5.5 Substitute Algorithm.....	13
6. Test Cases.....	18
TestNumberCard.java Test Cases.....	18
TestSkipCard.java Test Cases.....	19
TestReverseCard.java Test Cases.....	20
TestDrawTwoCard.java Test Cases.....	21
TestWildCard.java Test Cases.....	22
TestWildDrawFourCard.java Test Cases.....	24
TestUNOController Test Cases.....	25
TestHumanPlayer.java Test Cases.....	31
TestCPUPlayer.java Test Cases.....	32
TestConcreteCardFactory.java Test Cases.....	34
TestUNOGamePanel.java Test Cases.....	35
TestWildCardViewer.java Test Cases.....	39
TestChallangeViewer.java Test Cases.....	41
TestCountDownTimer.java Test Cases.....	42

1. Hierarchy Diagram



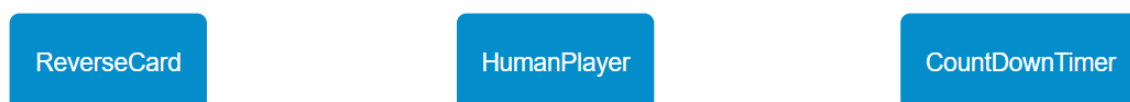
2. Testing Approach

We used a bottom-up approach for our testing process. We first conduct unit tests on methods of classes at the bottom of the hierarchy. Since they don't have any dependencies, they can be tested in isolation. Then, we identified and grouped classes into different components, such as Card, Player, and GamePanel. These are the sections of the game operation which should be tested as a group, as they have methods which are interconnected. We conducted integration testing by integrating the above-mentioned related components, such as card type classes (e.g., WildDrawFourCard, SkipCard), with the Card superclass. Finally, we conducted system testing to ensure the game can be run without errors.

We believe that by using this approach, we can rely less on test stubs, which improves testing efficiency. Additionally, since some classes are independent, different testers can test different components simultaneously, saving time without compromising coverage.

3. Testing Process

Our testing process is as follows:

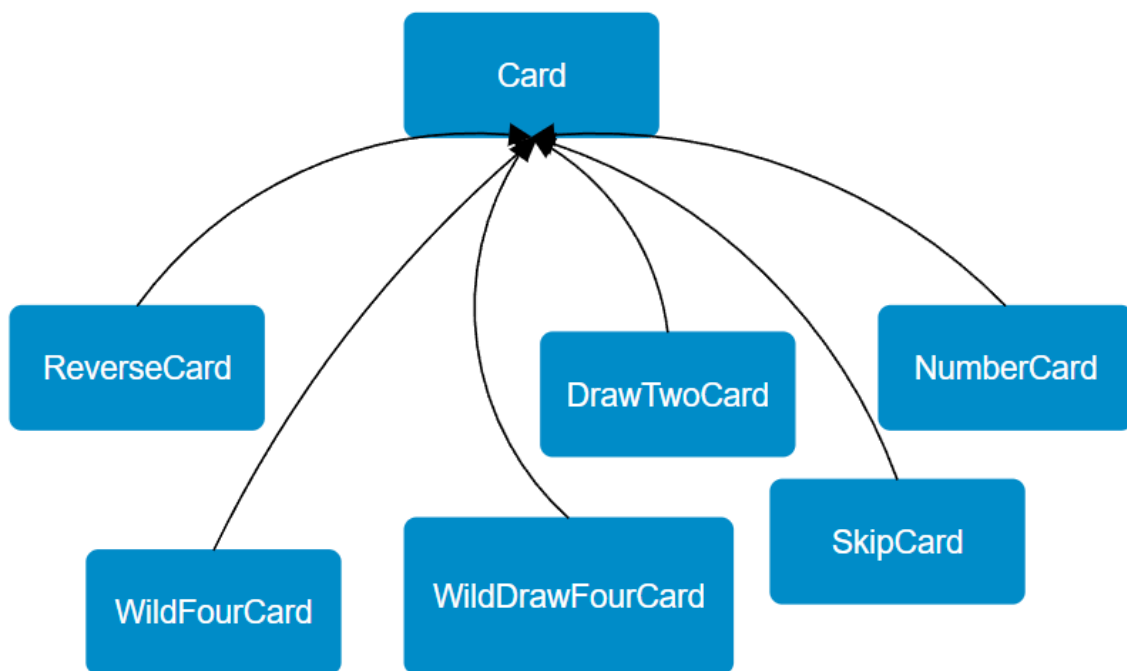


First, we conducted unit tests on the methods in the classes at the bottom of the hierarchy:

- ReverseCard
- WildFourCard
- WildDrawFourCard
- DrawTwoCard

- SkipCard
- NumberCard
- CountdownTimer
- HumanPlayer
- CPUPlayer
- ConcreteCardFactory
- DeckPlayCardViewer
- ResultViewer
- WildCardViewer
- ChallengeViewer

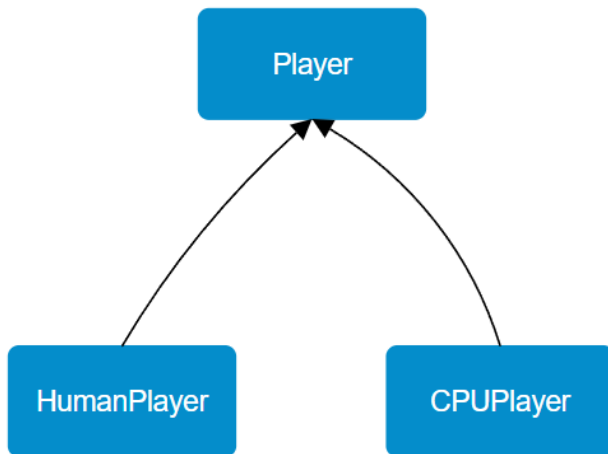
Next, we conduct integration testing, we broke it into 3 parts that mostly share the same properties like value and partial methods:



Card components:

- Card + ReverseCard + WildFourCard + WildDrawFourCard + DrawTwoCard + SkipCard + NumberCard

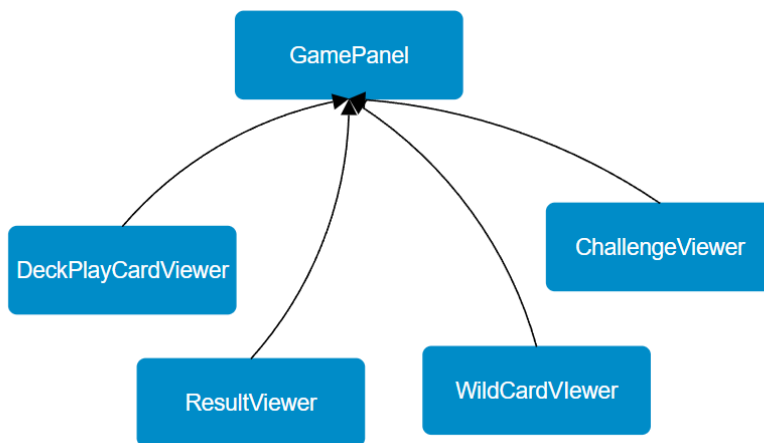
In these components, we mostly test on their unique methods for each type of card like `canPlayCard()`, `cardFunction()` to ensure the core logic of the game is correct. Additionally, since the abstract class `Card` also shares a variable for the reference to the `gamePanel` to paint the card, we also test some of its variables, such as `toString()`, as well as their `x` and `y` coordinates.



Player components:

- Player + HumanPlayer + CPUPlayer

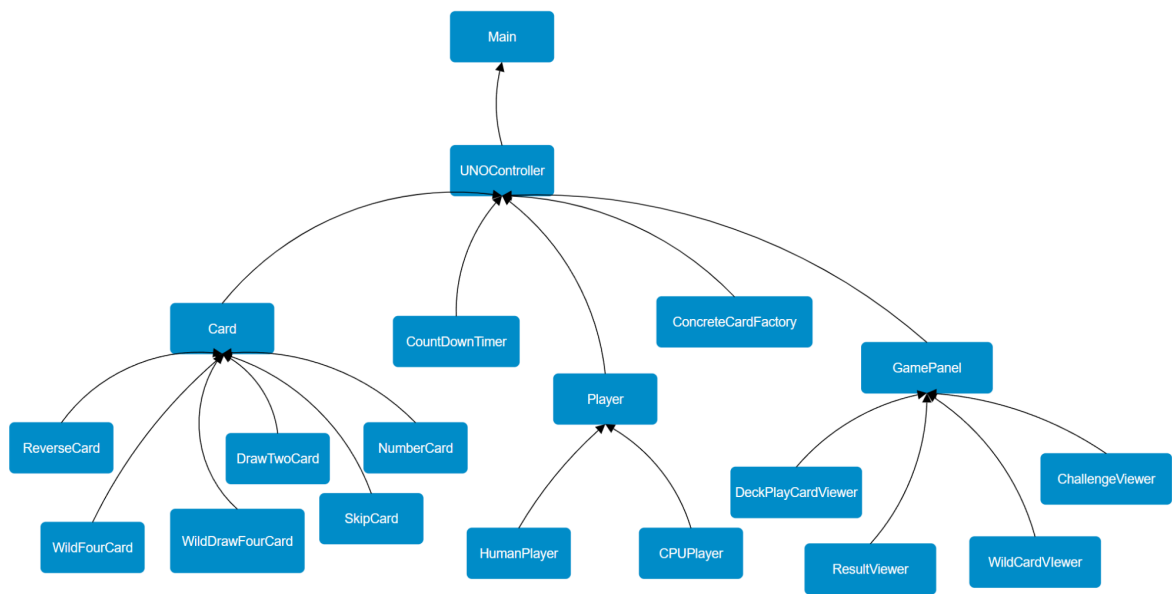
In these components, HumanPlayer mostly has to interact with the Viewer, which is the User interface, so all the methods of actions will be different from CPUPlayer, so we have to test on each Player type's methods to ensure the methods do not mix up. For example, CPUPlayer should not execute HumanPlayer's method accidentally, which may call the viewer accidentally display messages like "Challenge", "Choose to Play a Card" while it is not the human player's turn.



GamePanel components:

- GamePanel + DeckPlayCardViewer + ResultViewer + WildCardViewer + ChallengeViewer

Every viewer component is very important since it is the way for players to interact with the core game logic. If there are any errors on any buttons, it may affect players' experience negatively.



Lastly, system testing is completed, involving all classes, ensuring the game runs without any errors.

4. Coverage Analysis

CS3343-Group19-UNO		97.4 %	15,202	402	15,604
src		97.4 %	15,202	402	15,604
view		91.9 %	2,212	195	2,407
UNOGamePanel.java		91.9 %	1,267	112	1,379
DeckPlayCardViewer.java		74.1 %	140	49	189
WildCardViewer.java		93.8 %	423	28	451
ChallengeViewer.java		97.8 %	268	6	274
ResultViewer.java		100.0 %	114	0	114
model		95.7 %	2,315	105	2,420
HumanPlayer.java		94.7 %	584	33	617
ConcreteCardFactory.java		89.7 %	210	24	234
CPUPlayer.java		96.7 %	521	18	539
Card.java		94.0 %	236	15	251
WildCard.java		85.9 %	61	10	71
WildDrawFourCard.java		97.3 %	110	3	113
DrawTwoCard.java		98.2 %	108	2	110
CardFactory.java		100.0 %	10	0	10
Color.java		100.0 %	44	0	44
CountDownTimer.java		100.0 %	133	0	133
NumberCard.java		100.0 %	62	0	62
Player.java		100.0 %	54	0	54
ReverseCard.java		100.0 %	57	0	57
SkipCard.java		100.0 %	51	0	51
Type.java		100.0 %	74	0	74
test		99.3 %	9,514	64	9,578
TestUNOController.java		97.0 %	1,400	44	1,444
TestUNOGamePanel.java		99.5 %	1,305	7	1,312
TestCard.java		99.0 %	583	6	589
TestCountDownTimer.java		99.0 %	293	3	296
TestMain.java		99.6 %	492	2	494
TestCPUPlayer.java		99.8 %	631	1	632
TestDrawTwoCard.java		99.7 %	314	1	315
TestChallengeViewer.java		100.0 %	644	0	644
TestConcreteCardFactory.java		100.0 %	630	0	630
TestHumanPlayer.java		100.0 %	656	0	656
TestNumberCard.java		100.0 %	279	0	279
TestReverseCard.java		100.0 %	243	0	243
TestSkipCard.java		100.0 %	294	0	294
TestWildCard.java		100.0 %	276	0	276
TestWildCardViewer.java		100.0 %	1,154	0	1,154
TestWildDrawFourCard.java		100.0 %	320	0	320
controller		96.8 %	1,161	38	1,199
UNOController.java		97.1 %	1,156	35	1,191
Main.java		62.5 %	5	3	8

Module-Level Analysis

High Coverage Modules (>95%)

- Overall project (CS3343-Group19-UNO): 97.4% (15,202/15,604 instructions)
- src/test: 99.3% (9,514/9578, instructions)
- src/main/java model package: 95.7% (2,315/2,420 instructions)
- controller.UNOController.java: 97.1% (1,156/1,191 instructions)

Notable Coverage Areas

- TestUNOGamePanel.java: 99.5% (8,819/8,948)
- Multiple test files with 100% coverage, including:
 - TestNumberCard.java
 - TestSkipCard.java
 - TestReverseCard.java
 - TestWildCard.java
 - TestWildDrawFourCard.java
 - TestChallengeViewer.java
 - TestConcreteCardFactory.java
 - TestHumanPlayer.java

In model:

- Color.java, CountdownTimer.java, NumberCard.java, Player.java, ReverseCard.java, SkipCard.java all at 100% coverage

Lower Coverage Explanation

Main.java – 62.5%:

1. Mainly responsible for application startup and environment setup
2. Often only partially covered because tests bypass the full app bootstrap
3. Contains simple wiring/launch code that may not justify heavy testing effort

WildCard.java – 85.9% and HumanPlayer.java – 94.7%:

1. Include more complex behaviour branches and decision logic
2. Some edge cases or rare paths (e.g., special card effects, user decisions) may not be fully exercised by tests

5. Code Refactoring

Regarding the code refactoring aspect of our project, we frequently used 3 methods to improve code readability and modularization. The three methods include: Renaming and Merging Duplicate Code, Extract Method and Substitute Algorithm. Our code refactoring puts emphasis on model and controller components such as Cards and Players, as they contain the core game logic and state management that are performance-critical and platform-independent.

5.1 Renaming Methods/Variables

When using this method, our main objective is to improve variable or method names for clarity.

As we were developing the program, there were many occasions where we overlooked the variables' names and just used what was convenient at that time. As a result, the next time other team mates look back at the code, it is difficult for them to understand the code. Therefore, we changed some method names and variables' names to be more readable and deliver what they actually mean.

In the following example, we used short forms when naming variables, which made us confused.

Before:

```
public class UNOController {
    private static Controller inst;
    private UNOGamePanel gp;
    private JFrame f;
    private HashMap<String, Integer> d;
    private CardFactory cf = new ConcreteCardFactory();
    private ArrayList<Card> pc = new ArrayList<>();
    private Player p;
    private ArrayList<Player> ps;
    private int dir;
    private CountdownTimer t;
    private boolean act = false;
    private boolean frz = false;
    private WildCardViewer wv;
    private ChallengeViewer cv;
    private ResultViewer rv;
    private DeckPlayCardViewer dv;
    private boolean dr = false;
    private final int SZ = 7;
}
```

After:

```
public class UNOController {
    private static UNOController instance;
    private UNOGamePanel gamePanel;
    private JFrame mainFrame;
    private HashMap<String, Integer> Deck;
    private CardFactory cardFactory = new ConcreteCardFactory();
    private ArrayList<Card> PlayedCard = new ArrayList<>();
    private Player currentPlayer;
    private ArrayList<Player> players;
    private int playDirection;
    private CountdownTimer turnTimer;
    private boolean isAction = false;
    private boolean isFreezed = false;
    private WildCardViewer wildCardViewer;
    private ChallengeViewer challengeViewer;
    private ResultViewer resultViewer;
    private DeckPlayCardViewer deckPlayCardViewer;
    private boolean isDraw = false;
    private final int INITHANDSIZE = 7;
}
```

Note: We used camelCase for naming fields and methods, which improve the readability of our code.

5.2 Pull Up Field

Pull up field means moving the same fields in subclasses to superclass, which can enhance the maintainability of the classes when the subclasses share the same variables or methods.

In the following example, CPUPlayer and HumanPlayer are put the same variables (name, hand, isShout) independently, which will worsen the readability of the code. After creating the parent class players, we are able to pull up all the same variables and methods to the Player class, which can be maintained conveniently when we want to make changes to the Player class and their common methods.

Before:

```
public class CPUPlayer extends Player {  
  
    private Random random;  
    protected String name;  
    protected ArrayList<Card> hand;  
    protected boolean isShout;  
  
    public CPUPlayer(String name) {  
        super();  
        this.name = name;  
        this.hand = new ArrayList<>();  
        this.isShout = false;  
        this.random = new Random();  
    }  
}
```

```
public class HumanPlayer extends Player {  
  
    private Random random;  
    protected String name;  
    protected ArrayList<Card> hand;  
    protected boolean isShout;  
  
    public HumanPlayer(String name) {  
        super();  
        this.name = name;  
        this.hand = new ArrayList<>();  
        this.isShout = false;  
        this.random = new Random();  
    }  
}
```

After:

```
public class CPUPlayer extends Player {  
  
    private Random random;  
  
    public CPUPlayer(String name) {  
        super(name);  
        this.random = new Random();  
    }  
}
```

```
public class HumanPlayer extends Player {  
  
    private Random random;  
  
    public HumanPlayer(String name) {  
        super(name);  
        this.random = new Random();  
    }  
}
```

```
public abstract class Player implements Drawable,  
    Playable, Shoutable, Catchable, Challengeable {  
  
    protected String name;  
    protected ArrayList<Card> hand;  
    protected boolean isShout;  
    protected UNOController controller;  
  
    public abstract String shoutUno();  
  
    public Player(String name) {  
        this.name = name;  
        this.hand = new ArrayList<>();  
        this.isShout = false;  
    }  
}
```

5.3 Decompose Conditional

Decompose conditional is a refactoring that breaks a long or complex conditional into smaller, well-named methods, which makes the decision logic easier to read, reuse, and test.

In the following example, some conditions and else blocks are too long in shoutUno(). Therefore, we moved checks like “not current player with more than one card”, “not current player with one card”, and “has playable card” into separate boolean methods, and extracted the UNO shout action into performShout(). This reduces duplication inside shoutUno(), clarifies each rule, and makes future changes easier because the logic is simplified into small, focused methods.

Before:

HumanPlayer.java

```

public String shoutUno() {
    String errorMessage = null;
    if (this != controller.getCurrentPlayer() && this.getHand().size() > 1) {
        errorMessage = "You have more than 1 card!";
    } else if (this != controller.getCurrentPlayer() && this.getHand().size() == 1) {
        isShout = true;
        System.out.println(name + " shouts UNO!");
    } else if (this.getIsShout()){
        errorMessage = "You shouted UNO already!";
    } else if (this.getHand().size() > 2) {
        errorMessage = "You have more than 2 card!";
    } else {
        boolean hasPlayableCard = false;
        for (Card c : this.getHand()){
            if(controller.canPlayCard(c, controller.getTopCard(1))){
                hasPlayableCard = true;
            }
        }
        if (!hasPlayableCard) {
            errorMessage = "You have no playable card!";
            return errorMessage;
        }
        isShout = true;
        System.out.println(name + " shouts UNO!");
    }
    return errorMessage;
}

```

After:

HumanPlayer.java

```

private boolean hasPlayableCard() {
    boolean hasPlayableCard = false;
    for (Card c : this.getHand()) {
        if (controller.canPlayCard(c, controller.getTopCard(index: 1))) {
            hasPlayableCard = true;
        }
    }
    return hasPlayableCard;
}

private boolean notCurrentPlayerWithMoreThanOneCard() {
    return this != controller.getCurrentPlayer() && this.getHand().size() > 1;
}

private boolean notCurrentPlayerWithOneCard() {
    return this != controller.getCurrentPlayer() && this.getHand().size() == 1;
}

private void performShout() {
    isShout = true;
    System.out.println(name + " shouts UNO!");
}

@Override
public String shoutUno() {
    if (this.getIsShout()){
        return "You shouted UNO already!";
    } else if (notCurrentPlayerWithMoreThanOneCard()) {
        return "You have more than 1 card!";
    } else if (notCurrentPlayerWithOneCard()) {
        performShout();
    } else if (this.getHand().size() > 2) {
        return "You have more than 2 card!";
    } else if (!hasPlayableCard()) {
        return "You have no playable card!";
    }
    performShout();
    return null;
}

```

5.4 Extract Method

This method finds a piece of code that does something specific, then moves it into a new method with a clear, descriptive name, leading to better readability and reusability. Moreover, this avoids high coupling and maintenance issues which makes classes loosely connected and easy to change.

In the following example, we encapsulated the logic which checks if a card can be played or not in the `canPlayCard()` method, this method is then stored within the class of each type of card.

This approach avoids the controller operating directly on the value such as the color and number of cards to control the execution flow. Before this refactoring, our code contained many duplicate variable and function names such as various get and set methods. This made our code very difficult to understand and make changes because many methods have the same name. Therefore, extract method is implemented to mitigate this issue.

Before:

UNOController.java

```
public boolean canPlayCard(Card playedCard, Card matchCard) {
    if (matchCard == null) {
        return true;
    }

    switch (playedCard.getType()) {
        case Wild:
        case WildDrawFour:
            return true;
        case Skip:
        case Reverse:
        case DrawTwo:
            if (playedCard.getColor() == matchCard.getColor() || playedCard.getType() == matchCard.getType()) {
                return true;
            }
            return false;
        case Number:
            if (playedCard.getColor() == matchCard.getColor()) {
                return true;
            } else if (matchCard.getType() == Type.Number && ((NumberCard) playedCard).getValue() == ((NumberCard) matchCard).getValue()) {
                return true;
            }
            return false;
        default:
            return false;
    }
}
```

As you can see, we directly call the getter for type and color of cards which may cause high coupling, not a good design.

After:

UNOController.java

```
public boolean canPlayCard(Card matchCard, Card topCard) {
    if (topCard == null || matchCard == null) {
        return true;
    }

    return topCard.checkCard(matchCard);
}
```

Card.java

```
abstract public boolean checkCard(Card matchCard);
```

Note: We added an abstract method since every Card Type class needs to use the method.

ReverseCard.java

```
@Override
public boolean checkCard(Card playedCard){
    if (playedCard.type == Type.Wild || playedCard.type == Type.WildDrawFour) {
        return true;
    }
    if (playedCard.color == this.color || playedCard.type == this.type) {
        return true;
    }
    return false;
}
```

Note: Here we use ReverseCard as an example of checking card type.

5.5 Substitute Algorithm

This method is implemented to replace an entire algorithm or method body with a different, clearer, or more efficient one. The main effects that we want to achieve with this method are to improve performance and allow the game to be played smoothly on computers that have lower specifications.

In this example, we originally used a simple ArrayList to store the whole deck which has a high time complexity and has relatively poor performance for different Card operations of the ArrayList. After refactoring the code, we changed it to a hashmap. By using a hashmap instead of a simple ArrayList, the time complexity of some operations can be drastically reduced. For instance, we optimized the time complexity of removing cards, searching cards and counting cards from $O(N)$ to $O(1)$.

Before:

UNOController.java

```

private void initializeGame() {
    Deck = new ArrayList<String>(Arrays.asList(
        "r0", "r1", "r2", "r3", "r4", "r5", "r6", "r7", "r8", "r9",
        "r1", "r2", "r3", "r4", "r5", "r6", "r7", "r8", "r9",
        "g0", "g1", "g2", "g3", "g4", "g5", "g6", "g7", "g8", "g9",
        "g1", "g2", "g3", "g4", "g5", "g6", "g7", "g8", "g9",
        "b0", "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
        "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
        "y0", "y1", "y2", "y3", "y4", "y5", "y6", "y7", "y8", "y9",
        "y1", "y2", "y3", "y4", "y5", "y6", "y7", "y8", "y9",
        "rSkip", "rReverse", "rDrawTwo",
        "rSkip", "rReverse", "rDrawTwo",
        "gSkip", "gReverse", "gDrawTwo",
        "gSkip", "gReverse", "gDrawTwo",
        "bSkip", "bReverse", "bDrawTwo",
        "bSkip", "bReverse", "bDrawTwo",
        "ySkip", "yReverse", "yDrawTwo",
        "ySkip", "yReverse", "yDrawTwo",
        "WildCard", "WildCard", "WildCard", "WildCard",
        "WildDrawFour", "WildDrawFour", "WildDrawFour", "WildDrawFour"));

    // Clear played cards
    PlayedCard.clear();
    // Clear each player's hand
    for (Player player : players) {
        player.getHand().clear();
    }
    // Stop turn timer
    turnTimer.stopTimer();
    turnTimer.startTimer(30);
    gamePanel.setIsGameEnd(false);
    // Reset other game state variables if needed
    isAction = false;

    PlayedCard.add(cardFactory.giveCard(Deck, true, true, ""));

    for (int i = 0; i < INITCARDSIZE; i++) {
        players.get(0).drawCard(cardFactory.giveCard(Deck, false, true, ""));
        for (int j = 1; j < 4; j++) {
            players.get(j).drawCard(cardFactory.giveCard(Deck, false, false, ""));
        }
    }
    currentPlayer = players.get(0);
    System.out.println("Current Player: " + currentPlayer.getName());
    playDirection = 1;
}

```

ConcreteCardFactory.java

```

@Override
public Card createCard(ArrayList<String> Deck, Boolean isTop, boolean isRevealed, String cardString) {
    int Decklen = Deck.size();
    if(isTop){
        Decklen = 56;
    }
    String card;
    if (cardString != "" && Deck.indexOf(cardString) != -1) { // create specific card
        card = cardString;
        Deck.remove(Deck.indexOf(card));
    } else {
        if(cardString != ""){
            System.out.println("Deck doesn't contain this card!!!");
        }
        int index = random.nextInt(Decklen);
        card = Deck.get(index);
        Deck.remove(index);
    }

    if (card.equals("WildCard")){
        return new WildCard(isRevealed);
    } else if (card.equals("WildDrawFour")){
        return new WildDrawFourCard(isRevealed);
    } else {
        Color cardcolor;
        switch (card.charAt(0)){
            case 'r':
                cardcolor = Color.Red;
                break;
            case 'b':
                cardcolor = Color.Blue;
                break;
            case 'y':
                cardcolor = Color.Yellow;
                break;
            default:
                cardcolor = Color.Green;
                break;
        }
        switch (card.substring(1)) {
            case "Skip":
                return new SkipCard(cardcolor, isRevealed);
            case "Reverse":
                return new ReverseCard(cardcolor, isRevealed);
            case "DrawTwo":
                return new DrawTwoCard(cardcolor, isRevealed);
            default:
                return new NumberCard(cardcolor, Integer.parseInt(card.substring(1)), isRevealed);
        }
    }
}
}

```

Note: Apart from poorer performance, performing operations using ArrayList makes the code have code smell, which makes the code difficult to understand.

After:

UNOController.java

```

private void initializeGame() {
    Deck = new HashMap<>();
    Deck = initialDeck(Deck);

    // Clear played cards
    PlayedCard.clear();
    // Clear each player's hand
    for (Player player : players) {
        player.getHand().clear();
    }
    // Stop turn timer
    turnTimer.stopTimer();
    turnTimer.startTimer(seconds: 30);
    gamePanel.setIsGameEnd(isGameEnd: false);
    // Reset other game state variables if needed
    isAction = false;

    Card topCard = cardFactory.giveCard(Deck, isTop: true, isRevealed: true, cardString: "");
    if (topCard == null) {
        deckEmpty();
        return;
    }
    PlayedCard.add(topCard); // display the top card

    setAllPlayerCards();

    currentPlayer = players.get(index: 0);
    System.out.println("Current Player: " + currentPlayer.getName());
    playDirection = 1;
}

```

ConcreteCardFactory.java

```

@Override
public Card createCard(HashMap<String, Integer> Deck, Boolean isTop, boolean isRevealed, String cardString) {
    String card = null;

    if (Deck == null || Deck.isEmpty()) {
        return null;
    }

    if (isTop) {
        ArrayList<String> numberCards = getNumberCards(Deck);
        if (numberCards.isEmpty()) {
            return null;
        }
        card = getRandomCard(Deck, numberCards);
    } else if (!cardString.isEmpty() && Deck.containsKey(cardString) && Deck.get(cardString) > 0) { // create specific card
        card = cardString;
        removeCardFromDeck(Deck, card);
    } else {
        if (!cardString.isEmpty()) {
            System.out.println(x: "Deck doesn't contain this card!!!");
        }
        ArrayList<String> availableCards = new ArrayList<>(Deck.keySet());
        if (availableCards.isEmpty()) {
            return null;
        }
        card = getRandomCard(Deck, availableCards);
    }

    if (card == null) {
        return null;
    }

    return createSpecificCard(card, isRevealed);
}

```

Note: we also used “Extract Method” to improve code readability and consistency in ConcreteCardFactory.java, the above is the reference of the called method.

This method implements the mergeSort for different objects; the mergeSort does the divide-and-conquer algorithm, which maintains the relative order of equal elements. It is important for UNO card games to maintain certain order, like scores for players and card ordering.

In this example, the original code doesn’t have such a feature to order the cards, leading to a messy order and making cards difficult to find by just appending the card from the deck. Hence,

We use mergeSort to sort the players' cards in the players’ hands by comparing their card numbers, color priority, and type priority. Hence, it allows us to manage the order to display for the same classification of card, sort them by number, value, and type, and they will be put together for easier to

be seen, while it maintains $O(n \log n)$ time complexity to ensure the fast performance of sorting when drawing a card.

Original:

```
@Override
public void drawCard(Card card) {
    if (card != null) {
        this.hand.add(card);
    }
    isShout = false;
}
```

After:

```
@Override
public void drawCard(Card card) {
    if (card != null) {
        this.hand.add(card);
    }
    isShout = false;
    sortHandCards();
}
```

```
private void sortHandCards() {
    if (hand.size() <= 1) {
        return;
    }
    hand = mergeSort(hand);
}
```

```

private ArrayList<Card> mergeSort(ArrayList<Card> cards) {
    if (cards.size() <= 1) {
        return cards;
    }

    int mid = cards.size() / 2;
    ArrayList<Card> left = new ArrayList<>(cards.subList(0, mid));
    ArrayList<Card> right = new ArrayList<>(cards.subList(mid, cards.size()));

    left = mergeSort(left);
    right = mergeSort(right);

    return merge(left, right);
}

private ArrayList<Card> merge(ArrayList<Card> left, ArrayList<Card> right) {
    ArrayList<Card> result = new ArrayList<>();
    int i = 0, j = 0;

    while (i < left.size() && j < right.size()) {
        if (compareCards(left.get(i), right.get(j)) <= 0) {
            result.add(left.get(i));
            i++;
        } else {
            result.add(right.get(j));
            j++;
        }
    }

    while (i < left.size()) {
        result.add(left.get(i));
        i++;
    }

    while (j < right.size()) {
        result.add(right.get(j));
        j++;
    }

    return result;
}

```

```

private int compareCards(Card c1, Card c2) {
    if (c1 == null && c2 == null) return 0;
    if (c1 == null) return 1;
    if (c2 == null) return -1;

    int color1Priority = getColorPriority(c1);
    int color2Priority = getColorPriority(c2);

    if (color1Priority != color2Priority) {
        return Integer.compare(color1Priority, color2Priority);
    }

    return compareCardValue(c1, c2);
}

```

```

private int compareCardValue(Card c1, Card c2) {
    if (c1.getType() == Type.Wild || c1.getType() == Type.WildDrawFour) {
        return 0;
    }

    if (c1.getType() == Type.Number && c2.getType() == Type.Number) {
        return Integer.compare(c1.getValue(), c2.getValue());
    }

    if (c1.getType() == Type.Number) return -1;
    if (c2.getType() == Type.Number) return 1;

    int type1Priority = getTypePriority(c1);
    int type2Priority = getTypePriority(c2);

    return Integer.compare(type1Priority, type2Priority);
}

```

6. Test Cases

TestNumberCard.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
NumberCard_1	testNumberCardNotNull	NumberCard with color RED and value 5	Card created successfully with correct attributes	PASS
NumberCard_2	testNumberCardColor	Get the color of a NumberCard	Returns the correct color	PASS
NumberCard_3	testNumberCardValue	Get the value of a NumberCard	Returns the correct number value	PASS
NumberCard_4	testNumberCardType	Get Card Type	Returns NUMBER	PASS
NumberCard_5	testNumberCardToStringRed5	toString method	Returns "Red 5"	PASS
NumberCard_6	testNumberCardFunctionPassesNextPlayer	Number Card's card function will help to pass to the next player	The current player will change to the next player	PASS
NumberCard_7	testNumberCardFunctionWrapAround	Number Card's card function will help to pass back to the first player	The current player will change to the first player	PASS
NumberCard_8	testCheckCardSameColor	Number Card checking on the	Returns True	PASS

		same color		
NumberCard_9	testCheckCardSameNumber	Number Card checking on the same number	Returns True	PASS
NumberCard_10	testCheckCardWildCard	Number Card checking on wild card	Returns True	PASS
NumberCard_11	testCheckCardWildDrawFour	Number Card checking on wild draw 4 card	Returns True	PASS
NumberCard_12	testCheckCardDifferentColorAndNumber	Number Card checking on Number Card with a different number and color	Returns False	PASS
NumberCard_13	testCheckCardSkipDifferentColor	Number Card checking on Card with a different type(skip) and a different color	Returns False	PASS

TestSkipCard.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
SkipCard_1	testSkipCardNotNull	SkipCard with color BLUE	Card created successfully	PASS
SkipCard_2	testSkipCardColor	Get color of a SkipCard	Returns correct color	PASS
SkipCard_3	testSkipCardType	Get card type	Returns SKIP	PASS
SkipCard_4	testToString	toString method	Returns correct string representation	PASS
SkipCard_5	testSkipCardFunction	Skip Card function	the current player will become the third player	PASS
SkipCard_6	testSkipCardWithWrapAround	Current player is the last player in the list, play a Red Skip card	Next player should be the second player in the list (wrapping around)	PASS

SkipCard_7	testCheckCardSameColor	Top card is Red Skip, played card is Red Number 5	Returns True	PASS
SkipCard_8	testCheckCardSameType	Top card is Red Skip, played card is Blue Skip	Returns True	PASS
SkipCard_9	testCheckCardWildCard	Top card is Red Skip, played card is a Wild Card	Returns True	PASS
SkipCard_10	testCheckCardWildDrawFour	Top card is Red Skip, played card is a Wild Draw Four Card	Returns True	PASS
SkipCard_11	testCheckCardDifferentColorAndType	Top card is Red Skip, played card is Blue Reverse	Returns False	PASS
SkipCard_12	testCheckCardNumberDifferentColor	Top card is Red Skip, played card is Blue Number 5	Returns False	PASS

TestReverseCard.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
ReverseCard_1	testReverseCardNotNull	ReverseCard with color GREEN	Card created successfully	PASS
ReverseCard_2	testReverseCardColor	Get color of a ReverseCard	Returns correct color	PASS
ReverseCard_3	testReverseCardType	Get card type	Returns REVERSE	PASS
ReverseCard_4	testReverseCardValue	Create a Yellow Reverse card	Returns 20	PASS
ReverseCard_5	testReverseCardToString	Green Reverse card	Returns "Green Reverse"	PASS
ReverseCard_6	testReverseCardFunctionReversesDirection	Play direction is 1 (forward), play a Red Reverse card	Play direction should change to -1 (backward)	PASS
ReverseCard_7	testReverseCardFunctionReversesFromNegative	Play direction is -1 (backward), play a Red	Play direction should change to 1 (forward)	PASS

		Reverse card		
ReverseCard_8	testCheckCardSameColor	Top card is Red Reverse, played card is Red Number 5	Returns True	PASS
ReverseCard_9	testCheckCardSameType	Top card is Red Reverse, played card is Blue Reverse	Returns True	PASS
ReverseCard_10	testCheckCardWildCard	Top card is Red Reverse, played card is a Wild Card	Returns True	PASS
ReverseCard_11	testCheckCardWildDrawFour	Top card is Red Reverse, played card is a Wild Draw Four Card	Returns True	PASS
ReverseCard_12	testCheckCardDifferentColorAndType	Top card is Red Reverse, played card is Blue Skip	Returns False	PASS
ReverseCard_13	testCheckCardNumberDifferentColor	Top card is Red Reverse, played card is Blue Number 5	Returns False	PASS

TestDrawTwoCard.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
DrawTwoCard_1	testDrawTwoCardNotNull	Red Draw Two card	Card created successfully	PASS
DrawTwoCard_2	testDrawTwoCardColor	Get color of a DrawTwoCard	Returns correct color	PASS
DrawTwoCard_3	testDrawTwoCardType	Red Draw Two card	Card type should be DrawTwo	PASS
DrawTwoCard_4	testDrawTwoCardValue	Red Draw Two card	Card value should be 20	PASS
DrawTwoCard_5	testDrawTwoFunctionAddsCards	Target player has 1 card initially, current player plays a Red Draw	Target player should have 3 cards (initial + 2 drawn cards)	PASS

		Two card		
DrawTwoCard_6	testDrawTwoFunctionSkipsPlayer	Current player is player at index 1, play a Red Draw Two card	Current player should become player at index 3 (skipping the next player)	PASS
DrawTwoCard_7	testCheckCardSameColor	Top card is Red Draw Two, played card is Red Number 5	Returns True	PASS
DrawTwoCard_8	testCheckCardSameType	Top card is Red Draw Two, played card is Blue Draw Two	Returns True	PASS
DrawTwoCard_9	testCheckCardWildCard	Top card is Red Draw Two, played card is a Wild Card	Returns True	PASS
DrawTwoCard_10	testCheckCardWildDrawFour	Top card is Red Draw Two, played card is a Wild Draw Four Card	Returns True	PASS
DrawTwoCard_11	testCheckCardDifferentColorAndType	Top card is Red Draw Two, played card is Blue Skip	Returns False	PASS
DrawTwoCard_12	testCheckCardNumberDifferentColor	Top card is Red Draw Two, played card is Blue Number 5	Return False	PASS
DrawTwoCard_13	testDrawTwoWithEmptyDeck	Deck is empty, play a Draw Two card targeting next player	Target player's hand size should be at most initial size plus 2 cards	PASS

TestWildCard.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
WildCard_1	testWildCardNotNull	WildCard	Card created successfully and should not be null	PASS

WildCard_2	testWildCardColorNull	Wild Card	Card color should be null (not set initially)	PASS
WildCard_3	testWildCardType	Wild Card	Card type should be Wild	PASS
WildCard_4	testWildCardValue	Wild Card	Card value should be 50	PASS
WildCard_5	testWildCardToString	Wild Card	String representation should be "Wild Card"	PASS
WildCard_6	testWildCardSetColor	Wild Card and set its color to Red	Card color should be Red	PASS
WildCard_7	testWildCardSetColorBlue	Wild Card and set its color to Blue	Card color should be Blue	PASS
WildCard_8	testWildCardFunctionWithCPU	CPU player (index 1) plays a Wild Card	plays a Wild Card color should be automatically set (not null)	PASS
WildCard_9	testWildCardFunctionWithHuman	Human player (index 0) plays a Wild Card	Wild Card Viewer should display the card for color selection	PASS
WildCard_10	testCheckCardNumberCard	Top card is Wild (Red), played card is Red Number 5	Returns True	PASS
WildCard_11	testCheckCardSkipCard	Top card is Wild (Blue), played card is Blue Skip	Returns True	PASS
WildCard_12	testCheckCardReverseCard	Top card is Wild (Green), played card is Green Reverse	Returns True	PASS
WildCard_13	testCheckCardDrawTwoCard	Top card is Wild (Yellow), played card is Yellow Draw Two	Returns True	PASS
WildCard_14	testCheckCardWildCard	Top card is Wild, played card is another Wild Card	Returns True	PASS

WildCard_15	testCheckCardWildDrawFour	Top card is Wild, played card is Wild Draw Four	Returns True	PASS
-------------	---------------------------	---	--------------	------

TestWildDrawFourCard.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
WildDrawFour_1	testWildDrawFourCardNotNull	Wild Draw Four card	Card object should not be null	PASS
WildDrawFour_2	testWildDrawFourCardColorNull	Wild Draw Four card	Card color should be null (not set initially)	PASS
WildDrawFour_3	testWildDrawFourCardType	Wild Draw Four card	Card type should be WildDrawFour	PASS
WildDrawFour_4	testWildDrawFourCardValue	Wild Draw Four card	Card value should be 50	PASS
WildDrawFour_5	testWildDrawFourCardToString	Wild Draw Four card	String representation should be "Wild Draw Four"	PASS
WildDrawFour_6	testWildDrawFourCardSetColor	Wild Draw Four card and set its color to Red	Card color should be Red	PASS
WildDrawFour_7	testWildDrawFourCardFunctionWithHuman	Human player (index 0) plays a Wild Draw Four card	Wild Card Viewer should display a card (not null) for color selection	PASS
WildDrawFour_8	testWildDrawFourCardFunctionSetColor	CPU player (index 1) plays a Wild Draw Four card with play direction -1	Card color should be automatically set (not null)	PASS
WildDrawFour_9	testCheckCardNumberCard	Top card is Wild Draw Four (Red), played card is Red Number 5	Returns True	PASS
WildDrawFour_10	testCheckCardSkipCard	Top card is Wild Draw Four (Blue), played	Returns True	PASS

		card is Blue Skip		
WildDrawFour_1 1	testCheckCardReverseCard	Top card is Wild Draw Four (Green), played card is Green Reverse	Returns True	PASS
WildDrawFour_1 2	testCheckCardDrawTwoCard	Top card is Wild Draw Four (Yellow), played card is Yellow Draw Two	Return True	PASS
WildDrawFour_1 3	testCheckCardWildCard	Top card is Wild Draw Four, played card is Wild Card	Returns True	PASS
WildDrawFour_1 4	testCheckCardWildDrawFour	Top card is Wild Draw Four, played card is another Wild Draw Four	Returns True	PASS
WildDrawFour_1 5	testWildDrawFourWithEmptyDeck	Deck is empty, player plays Wild Draw Four targeting next player	Next player's hand size should remain unchanged since deck is empty	PASS

TestUNOController Test Cases

TestID	Test Name	Input / Description	Expected	Status
UNOController_1	testGetInstance	Call getInstance twice	Both calls should return the same singleton instance	PASS
UNOController_2	testResetInstance	Get instance, reset, then get instance again	Second instance should be different from first instance	PASS
UNOController_3	testSetPlayers	Start game and get player list	Player list should have 4 players	PASS
UNOController_4	testFirstPlayerIsHuman	Check player at index 0	Player should be an instance of HumanPlayer	PASS

UNOController_5	testSecondPlayerIsCPU	Check player at index 1	Player should be an instance of CPUPlayer	PASS
UNOController_6	testThirdPlayerIsCPU	Check player at index 2	Player should be an instance of CPUPlayer	PASS
UNOController_7	testFourthPlayerIsCPU	Check player at index 3	Player should be an instance of CPUPlayer	PASS
UNOController_8	testHumanPlayerName	Get name of player at index 0	Player name should be "Player"	PASS
UNOController_9	testCPU1Name	Get name of player at index 1	Player name should be "CPU1"	PASS
UNOController_10	testCPU2Name	Get name of player at index 2	Player name should be "CPU2"	PASS
UNOController_11	testCPU3Name	Get name of player at index 3	Player name should be "CPU3"	PASS
UNOController_12	testStartGameSetsCurrentPlayer	Start game and check current player	Current player should not be null	PASS
UNOController_13	testStartGameFirstPlayerCurrent	Start game and check current player	Current player should be first player (index 0)	PASS
UNOController_14	testStartGamePlayDirection	Start game and check play direction	Play direction should be 1 (clockwise)	PASS
UNOController_15	testStartGameInitializesDeck	Start game and get deck	Deck should not be null	PASS
UNOController_16	testStartGameSetsTopCard	Start game and get top card	Top card should not be null	PASS
UNOController_17	testSetAndGetCurrentPlayer	Set current player to player at index 2	getCurrentPlayer should return player at index 2	PASS
UNOController_18	testSetPlayDirectionClockwise	Set play direction to 1	Play direction should be 1	PASS
UNOController_19	testSetPlayDirectionCounterClockwise	Set play direction to -1	Play direction should be -1	PASS
UNOController_20	testCheckCurrentPlayerFirst	Set current player to first player	checkCurrentPlayer should return 0	PASS
UNOController_21	testCheckCurrentPlayerThird	Set current player to third player	checkCurrentPlayer should return 2	PASS

UNOController_22	testCheckPlayerFirst	Check index of first player	Should return 0	PASS
UNOController_23	testCheckPlayerSecond	Check index of second player	Should return 1	PASS
UNOController_24	testCheckPlayerFourth	Check index of fourth player	Should return 3	PASS
UNOController_25	testPassNextPlayerClockwise	Current player is first, direction is clockwise, pass to next	Current player should become second player	PASS
UNOController_26	testPassNextPlayerClockwiseTwice	Current player is first, direction is clockwise, pass twice	Current player should become third player	PASS
UNOController_27	testPassNextPlayerCounterClockwise	Current player is first, direction is counter-clockwise, pass to next	Current player should become fourth player (wrapping around)	PASS
UNOController_28	testPassNextPlayerCounterClockwiseTwice	Current player is first, direction is counter-clockwise, pass twice	Current player should become third player	PASS
UNOController_29	testPassNextPlayerWrapsAround	Current player is fourth, direction is clockwise, pass to next	Current player should become first player (wrapping around)	PASS
UNOController_30	testPassNextPlayerWithSkip	Current player is first, direction is clockwise, pass with skip value 2	Current player should become third player	PASS
UNOController_31	testCanPlayCardMatchingColor	Top card is Red 5, play card is Red 3	Returns true	PASS

UNOController_32	testCanPlayCardMatchingNumber	Top card is Red 5, play card is Blue 5	Returns true	PASS
UNOController_33	testCanPlayCardWildCard	Top card is Red 5, play Wild Card set to Red	Returns true	PASS
UNOController_34	testCanPlayCardWildDrawFour	Top card is Red 5, play Wild Draw Four set to Red	Returns true	PASS
UNOController_35	testCanPlayCardNonMatching	Top card is Red 5, play card is Blue 3	Returns false	PASS
UNOController_36	testCanPlayCardMatchingActionCards	Top card is Red Skip, play card is Blue Skip	Returns true	PASS
UNOController_37	testCanPlayCardNullTopCard	Top card is null, play any card	Returns true	PASS
UNOController_38	testPlayCard	Play Red 5 card	Top card should be the played Red 5 card	PASS
UNOController_39	testPlayCardRemovesFromHand	Current player has Red 5 in hand, play the card	Player's hand size should be 0 (card removed)	PASS
UNOController_40	testSetIsFreezedTrue	Set isFreezed to true	getIsFreezed should return true	PASS
UNOController_41	testSetIsFreezedFalse	Set isFreezed to false	getIsFreezed should return false	PASS
UNOController_42	testGetCardFactory	Get card factory	Card factory should not be null	PASS
UNOController_43	testGetCardFactoryType	Get card factory	Card factory should be instance of CardFactory	PASS
UNOController_44	testGetDeck	Get deck	Deck should not be null	PASS
UNOController_45	testGetDeckHasCards	Get deck	Deck size should be greater than 0	PASS
UNOController_46	testGetTurnTimer	Get turn timer	Turn timer should not be null	PASS
UNOController_47	testGetWildCardViewer	Get wild card viewer	Wild card viewer should not be null	PASS
UNOController_48	testGetChallengeViewer	Get challenge viewer	Challenge viewer should not be null	PASS

UNOController_49	testGetResultViewer	Get result viewer	Result viewer should not be null	PASS
UNOController_50	testGetDeckPlayCardViewer	Get deck play card viewer	Deck play card viewer should not be null	PASS
UNOController_51	testGetGamePanel	Get game panel	Game panel should not be null	PASS
UNOController_52	testGetHumanCard	Get player card for index 0	Should return first player's hand	PASS
UNOController_53	testGetCPU1Card	Get player card for index 1	Should return second player's hand	PASS
UNOController_54	testGetCPU2Card	Get player card for index 2	Should return third player's hand	PASS
UNOController_55	testIsGameEnd	Current player has no cards, play matching card	Should return true (game ends)	PASS
UNOController_56	testIsGameEndWithCards	Current player has cards in hand, play card	Should return false (game continues)	PASS
UNOController_57	testGetSortedPlayersScoreNotNull	Get sorted players by score	Sorted player list should not be null	PASS
UNOController_58	testGetSortedPlayersScoreSize	Get sorted players by score	Sorted player list should have 4 players	PASS
UNOController_59	testGetSortedPlayersScoreCurrentFirst	Current player is first, get sorted players	Current player should be first in sorted list	PASS
UNOController_60	testGetCardFromDeck	Current player gets card from deck	Player's hand size should increase by 1	PASS
UNOController_61	testCanPlayCardSkipMatchingColor	Top card is Red Skip, play Red 5	Returns true	PASS
UNOController_62	testCanPlayCardReverseMatchingType	Top card is Red Reverse, play Blue Reverse	Returns true	PASS
UNOController_63	testCanPlayCardDrawTwoMatchingType	Top card is Red Draw Two, play Blue Draw Two	Returns true	PASS

UNOController_64	testGetPlayerListNotNull	Get player list	Player list should not be null	PASS
UNOController_65	testGetPlayerListSize	Get player list	Player list should have 4 players	PASS
UNOController_66	testGetPlayerList	Get player list	Should return the same player list instance	PASS
UNOController_67	testGetTopCardMostRecent	Play Red 5 then Blue 3, get top card with index 1	Should return Blue 3 (most recent)	PASS
UNOController_68	testGetTopCardSecondRecent	Play Red 5 then Blue 3, get top card with index 2	Should return Red 5 (second most recent)	PASS
UNOController_69	testDeckEmptySetsDraw	Call deckEmpty method	getIsDraw should return true	PASS
UNOController_70	testDrawFromEmptyDeck	Deck is empty, current player draws card	Hand size should remain unchanged and getIsDraw returns true	PASS
UNOController_71	testOnTimerCompleteDrawsCard	Timer completes after 1 second	Current player should have one more card in hand	PASS
UNOController_72	testOnTimerCompleteWithWildCard	Wild Card is pending, timer completes	Wild card should have color set and no longer be pending	PASS
UNOController_73	testOnTimerCompleteWithWildDrawFour	Wild Draw Four is pending, timer completes	Wild Draw Four should have color set and no longer be pending	PASS
UNOController_74	testOnTimerCompleteWithChallenge	Challenge is pending, timer completes	Challenge should auto-decline and next player draws 4 cards	PASS
UNOController_75	testOnTimerCompleteWithDeckCardDecision	Deck card decision is pending, timer completes	Deck viewer should no longer be in deciding state	PASS
UNOController_76	testOnTimerCompleteCallsGetCardFromDeck	Timer completes	Either card is drawn or player changes	PASS
UNOController_77	testOnTimerCompleteDefaultBehavior	Timer completes without special conditions	Some game action should occur (card drawn, player changed, or deck viewer active)	PASS

TestHumanPlayer.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
HumanPlayer_1	testGetName	Get name of human player	Returns "Player"	PASS
HumanPlayer_2	testDrawCardIncreasesHandSize	Draw one number card	Hand size +1	PASS
HumanPlayer_3	testDrawCardAddsToHand	Draw one number card	Card added to hand	PASS
HumanPlayer_4	testDrawNullCard	Draw null card	Hand size unchanged	PASS
HumanPlayer_5	testDrawCardResetsShout	Set shout flag true, then draw a card	Shout flag becomes false	PASS
HumanPlayer_6	testShoutUnoNull	One playable card, shout UNO	Returns null(no error message)	PASS
HumanPlayer_7	testShoutUnoSetsFlag	One playable card, shout UNO	Shout flag becomes true	PASS
HumanPlayer_8	testShoutUNOMoreThanOneCard	More than one card, shout UNO	Returns "You have more than 1 card!"	PASS
HumanPlayer_9	testShoutUNOTwice	Shout flag already true, shout again	Returns "You shouted UNO already!"	PASS
HumanPlayer_10	testShoutUNONoPlayableCard	One non-playable card, shout UNO	Returns "You have no playable card!"	PASS
HumanPlayer_11	testPlayCardRemovesFromHand	Play a card that is in hand	Card removed from hand	PASS
HumanPlayer_12	testPlayCardDoesNotSetShout	Play a card with more than one card in hand	Shout flag stays false	PASS

HumanPlayer_13	testPlayCardNotInHand	Play a card not in hand	Hand remains empty	PASS
HumanPlayer_14	testCatchForgotShoutSuccess	Target has 1 card, shout flag false	Target draws 2 cards (hand size 3)	PASS
HumanPlayer_15	testCatchForgotShoutAlreadyShouted	Target has 1 card, shout flag true	No penalty (hand size 1)	PASS
HumanPlayer_16	testCatchForgotShoutMultipleCards	Target has multiple cards, shout flag false	No penalty (hand size unchanged)	PASS
HumanPlayer_17	testChallengeDrawFourSuccess	Target has matching color card when challenged	Target draws 4 cards	PASS
HumanPlayer_18	testChallengeDrawFourFail	Target only has Wild Draw Four, challenger calls challenge	Challenger draws 6 cards	PASS
HumanPlayer_19	testShoutUnoEmptyHand	Empty hand, shout UNO	Returns "You have no playable card!"	PASS
HumanPlayer_20	testDrawCardFromEmptyDeck	Deck empty, try to draw via card factory and then draw that card	No card drawn; hand size unchanged	PASS

TestCPUPlayer.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
CPUPlayer_1	testDrawCard	CPU draws one red 5 card	Hand size increases by 1	PASS

CPUPlayer_2	testPlayCard	CPU has red 5, matching top red card, then plays red 5	Played card removed from hand	PASS
CPUPlayer_3	testShoutUnoTrue	CPU has exactly one card and shouts UNO	Shout flag becomes true	PASS
CPUPlayer_4	testShoutUnoFalse	CPU has more than one card and shouts UNO	Shout flag remains false	PASS
CPUPlayer_5	testCatchForgotShout	Target has 1 card and has not shouted	Target draws 2 penalty cards	PASS
CPUPlayer_6	testChallengeDrawFour_Challenge Fails	Previous player had no matching color; CPU challenges	Challenge fails, CPU draws 6 cards	PASS
CPUPlayer_7	testChallengeDrawFour_Challenge Succeeds	Previous player had matching color; CPU challenges	Challenge succeeds, target draws 4 cards	PASS
CPUPlayer_8	testChallengeDrawFour_notChallenge	Previous player has one card; situation does not trigger challenge	CPU draws 4 cards	PASS
CPUPlayer_9	testChooseCard_OnlyOnePlayable Card	Hand has one playable red 5 and one non-playable green 3	CPU plays red 5, hand size decreases by 1	PASS
CPUPlayer_10	testChooseColor_DominantColor	Hand contains mostly red cards and one blue card	CPU chooses Red as color	PASS
CPUPlayer_11	testCPUDrawFromEmptyDeck	Deck is empty and CPU attempts to draw	No card drawn; hand size unchanged	PASS

TestConcreteCardFactory.java Test Cases

TestID	Test Name	Input / Description	Expected	Status
ConcreteCardFactory_1	testCreateNumberCard	Deck contains r5, create card with isTop=false and revealed=true	Card should not be null, type is Number, color is Red, value is 5	PASS
ConcreteCardFactory_2	testCreateSkipCard	Deck contains bSkip, create card with isTop=false and revealed=true	Card should not be null, type is Skip, color is Blue	PASS
ConcreteCardFactory_3	testCreateReverseCard	Deck contains yReverse, create card with isTop=false and revealed=true	Card should not be null, type is Reverse, color is Yellow	PASS
ConcreteCardFactory_4	testCreateDrawTwoCard	Deck contains gDrawTwo, create card with isTop=false and revealed=true	Card should not be null, type is DrawTwo, color is Green	PASS
ConcreteCardFactory_5	testCreateWildCard	Deck contains WildCard, create card with isTop=false and revealed=true	Card should not be null, type is Wild	PASS
ConcreteCardFactory_6	testCreateWildDrawFourCard	Deck contains WildDrawFour, create card with isTop=false and revealed=true	Card should not be null, type is WildDrawFour	PASS
ConcreteCardFactory_7	testIsTopSelectsNumberCard	Deck contains r5, bSkip, and WildCard, create card with isTop=true	Card should not be null and type should be Number (only number cards selected when isTop=true)	PASS
ConcreteCardFactory_8	testIsTopWithOnlyNumberCards	Deck contains r0, b3, and y7, create card with isTop=true	Card should not be null and type should be Number	PASS
ConcreteCardFactory_9	testRevealedTrue	Deck contains r5, create card with revealed=true	Card's isRevealed should return true	PASS

ConcreteCardFactory_10	testRevealedFalse	Deck contains r5, create card with revealed=false	Card's isRevealed should return false	PASS
ConcreteCardFactory_11	testCreateCardReducesDeckSize	Deck contains 3 r5 cards, create one card	Deck should have 2 r5 cards remaining	PASS
ConcreteCardFactory_12	testCreateCardRemovesLastCard	Deck contains 1 r5 card, create one card	Deck should no longer contain r5 key	PASS
ConcreteCardFactory_13	testCreateMultipleCards	Deck contains 3 r5 cards, create two cards	Both cards should not be null, deck should have 1 r5 card remaining	PASS
ConcreteCardFactory_14	testCreateCardFromEmptyDeck	Deck is empty, create card	Card should be null	PASS
ConcreteCardFactory_15	testIsTopWithEmptyDeck	Deck is empty, create card with isTop=true	Card should be null	PASS
ConcreteCardFactory_16	testIsTopWithNoNumberCards	Deck contains only bSkip and WildCard, create card with isTop=true	Card should be null (no number cards available)	PASS
ConcreteCardFactory_17	testCreateRedCard	Deck contains r3, create card	Card color should be Red	PASS
ConcreteCardFactory_18	testCreateBlueCard	Deck contains b7, create card	Card color should be Blue	PASS
ConcreteCardFactory_19	testCreateYellowCard	Deck contains y2, create card	Card color should be Yellow	PASS
ConcreteCardFactory_20	testCreateGreenCard	Deck contains g9, create card	Card color should be Green	PASS
ConcreteCardFactory_21	testCreateNumber0	Deck contains r0, create card	Card value should be 0	PASS
ConcreteCardFactory_22	testCreateNumber9	Deck contains b9, create card	Card value should be 9	PASS
ConcreteCardFactory_23	testGiveCardNotNull	Deck contains r5, call giveCard method	Card should not be null	PASS
ConcreteCardFactory_24	testGiveCardReducesDeck	Deck contains 2 r5 cards, call giveCard method	Deck should have 1 r5 card remaining	PASS
ConcreteCardFactory_25	testGiveCardFromEmptyDeck	Deck is empty, call giveCard method	Card should be null	PASS

TestUNOGamePanel.java Test Cases

Test ID	Test Name	Input / Description	Expected	Status
UNOGamePanel _1	testGamePanelNotNull	Create new UNOGamePanel	Panel instance is not null	PASS
UNOGamePanel _2	testGamePanelVisible	Call setVisible(true)	isVisible() returns true	PASS
UNOGamePanel _3	testSetIsGameEndTrue	Call setIsGameEnd(true) and read private flag via reflection	isGameEnd is true	PASS
UNOGamePanel _4	testSetIsGameEndFalse	Call setIsGameEnd(false)	isGameEnd is false	PASS
UNOGamePanel _5	testInitialIsGameEndFalse	New panel, read isGameEnd	Initial isGameEnd is false	PASS
UNOGamePanel _6	testStartGameNotNull	Start controller game, call startGame() on panel	Panel remains valid (not null, no exception)	PASS
UNOGamePanel _7	testUpdateDisplayNotNull	Start game, call updateDisplay()	Panel remains valid	PASS
UNOGamePanel _8	testShoutUnoNotNull	Start game, call shoutUno()	Panel remains valid	PASS
UNOGamePanel _9	testSelectedCardWithValidIndex	Start game, player 0's turn, call selectedCard(0)	currentSelectedCardIndex becomes 0	PASS
UNOGamePanel _10	testSelectedCardResetsIndex	After selecting index 0, force index to -1 via helper	currentSelectedCardIndex is -1	PASS
UNOGamePanel _11	testCreateCardButtonNotNull	Create red 5 card at (100,100), call createCardButton	Returned JButton is not null	PASS

UNOGamePanel _12	testCreateCardButtonBounds	Same as above, check button x	Button x-coordinate is 100	PASS
UNOGamePanel _13	testUpdateCardButtonsNotNull	Start game, call updateCardButtons()	Panel remains valid	PASS
UNOGamePanel _14	testUnoButtonExists	Repaint and paint panel, inspect private UnoButton	UNO button exists (not null)	PASS
UNOGamePanel _15	testUnoButtonAction	Spy panel overrides shoutUno, click UnoButton	Overridden shoutUnocalled (flag set true)	PASS
UNOGamePanel _16	testDeckButtonWhenGameEnded	Get deck button via updateDeck, set game end, click button	Deck size unchanged	PASS
UNOGamePanel _17	testDeckButtonWhenNotPlayersTurn	Not player 0's turn, click deck button	errorMessage is "It's not your turn!"	PASS
UNOGamePanel _18	testCatchButtonWhenGameEnded	Game ended, click catchcpu1Button	No effect; test just completes successfully	PASS
UNOGamePanel _19	testCatchButtonCPU1Action	Game running, frozen, click catchcpu1Button	Button exists and action executes (not null)	PASS
UNOGamePanel _20	testCatchButtonCPU2Action	Game running, frozen, click catchcpu2Button	Button exists and action executes	PASS
UNOGamePanel _21	testCatchButtonCPU3Action	Game running, frozen, click catchcpu3Button	Button exists and action executes	PASS
UNOGamePanel _22	testSelectedCardWhenGameEnded	Game ended, call selectedCard(0)	Selected index unchanged	PASS

UNOGamePanel _23	testSelectedCard WhenNotPlayerTurn	Game running, current player is CPU, call selectedCard(0)	errorMessage is "It's not your turn!"	PASS
UNOGamePanel _24	testSelectedCard ClickUnselected Card	Player 0's turn, no card selected, call selectedCard(0)	Index set to 0; that card marked selected	PASS
UNOGamePanel _25	testSelectedCard ClickSelectedCardCannotPlay	Selected card is unplayable, click same index again	errorMessage is "Can't play this card!"	PASS
UNOGamePanel _26	testSelectedCard ClickAnotherCard	One card already selected at 0, click index 1	Card 0 deselected, card 1 selected, index becomes 1	PASS
UNOGamePanel _27	testErrorMessage DisplayWithinTimeout	Set errorMessageand recent errorMessageTimer, call paint	errorMessage still non-null after painting	PASS
UNOGamePanel _28	testErrorMessage ClearedAfterTimeout	Set errorMessageand old timer, call paint	errorMessage remains "Test Error" after painting	PASS
UNOGamePanel _29	testErrorMessage Null	errorMessage set to null, call paint	errorMessage remains null	PASS
UNOGamePanel _30	testShoutUnoSets ErrorMessage	Game running, frozen, shout UNO via panel	errorMessage becomes non-null	PASS
UNOGamePanel _31	testShoutUnoWhenGameEnded	Game ended, call shoutUno()	errorMessage unchanged	PASS
UNOGamePanel _32	testHasDeckButtonReturnsFalse	Check deck button before update	hasDeckButton() returns false	PASS
UNOGamePanel _33	testHasDeckButtonReturnsTrue	Start game, call updateCardButtons(), then check	hasDeckButton() returns true	PASS

TestWildCardViewer.java Test Cases

Test ID	Test Name	Input / Description	Expected Outcome	Actual Outcome
WildCardViewer_1	testNoWildCardState()	Testing default state after initialization of UNOController	isHavingWild should be False	PASS
WildCardViewer_2	testSetWildCardSetsFlag()	Initialization of UNOController and wild card set into wildCardViewer	isHavingWild should be True	PASS
WildCardViewer_3	testGetCardReturnsNull()	Test obtaining stored wildcard during default state	getCard() method should return null	PASS
WildCardViewer_4	testEndTimeAutoSelectColorSetsRed()	Simulated running out of countdown time by executing autoSelectColor() method	Wild card color should automatically be set to red	PASS
WildCardViewer_5	testDrawButtonsCreatesRedButton()	Simulated wild card state to check whether wild card color selection buttons are viewable	Assert test should confirm that selection buttons are visible to the player	PASS
WildCardViewer_6	testDrawButtonsCreatesBlueButton()			PASS
WildCardViewer_7	testDrawButtonsCreatesYellowButton()			PASS
WildCardViewer_8	testDrawButtonsCreatesGreenButton()			PASS

WildCardViewer_9	testRemoveButtonsClearsRedButton()	Simulated that wild card color selection buttons are removed after a button is selected	Assert test should confirm that selection buttons are not visible to the player	PASS
WildCardViewer_10	testRemoveButtonsClearsBlueButton()			PASS
WildCardViewer_11	testRemoveButtonsClearsYellowButton()			PASS
WildCardViewer_12	testRemoveButtonsClearsGreenButton()			PASS
WildCardViewer_13	testRedButtonClickSetsRedColor()	Simulated selecting wild card color selection to change its color state	Wild card color should be changed to the respective selected color	PASS
WildCardViewer_14	testBlueButtonClickSetsBlueColor()			PASS
WildCardViewer_15	testYellowButtonClickSetsYellowColor()			PASS
WildCardViewer_16	testGreenButtonClickSetsGreenColor()			PASS
WildCardViewer_17	testRedButtonClickResetsHavingWild()	Simulated selecting any wild card color to change its state	wildState should return to false	PASS
WildCardViewer_18	testBlueButtonClickResetsHavingWild()			PASS
WildCardViewer_19	testYellowButtonClickResetsHavingWild()			PASS
WildCardViewer_20	testGreenButtonClickResetsHavingWild()			PASS

WildCardViewer_21	testCountdownTimerVisibleDuringWildCardRound()	Simulated wild card state to check whether countdown timer is viewable	Assert test should confirm that countdown timer visible to the player	PASS
WildCardViewer_22	testCountDownTimerHiddenBeforeWildCardRound()	Checking whether countdown timer is visible during default state	Countdown Timer should not be visible	PASS
WildCardViewer_23	testCountDownTimerHiddenAfterWildCardRound()	Checking whether countdown timer is visible after returning to default state	Countdown Timer should not be visible	PASS

TestChallengeViewer.java Test Cases

Test ID	Test Name	Input / Description	Expected Outcome	Actual Outcome
ChallengeViewer_1	testInitialStateNoChallenging()	Testing default state after initialization of ChallengeViewer	ChallengeViewer should not be in challenge state	PASS
ChallengeViewer_2	testSetChallengeSetsFlag()	Simulated challenge state being true	getIsChallenging() method should return True	PASS
ChallengeViewer_3	testDrawButtonsCreatesRedButton()	Simulated wild card state to check whether both option's buttons are viewable	Assert test should confirm that selection buttons are visible to the player	PASS
ChallengeViewer_4	testDrawButtonsCreatesGreenButton()			PASS
ChallengeViewer_5	testRemoveButtonsClearsRedButton()	Simulated that both selection buttons are	Assert test should confirm that selection buttons	PASS

ChallangeViewer _6	testRemoveButtonsClearsGreenButton()	removed after a button is selected	are not visible to the player	PASS
ChallangeViewer _7	testRedButtonClickAddsCards()	Tested that choosing to not challenge adds cards to challenger player	An additional four cards should be added to the player's card hand	PASS
ChallangeViewer _8	testRedButtonClickResetsChallenging()	Clicking either selection button should exit challenge state	Challenge viewer should not be in challenge state	PASS
ChallangeViewer _9	testGreenButtonClickResetsChallenging()			PASS

TestCountDownTimer.java Test Cases

Test ID	Test Name	Input / Description	Expected Outcome	Actual Outcome
CountDownTimer r_1	testTimerStart()	To test starting countdown timer	timerIsActive() and isRunning() methods should return true	PASS
CountDownTimer r_2	testTimerStop()	To test stopping countdown timer	timerIsActive() and isRunning() methods should return false	PASS
CountDownTimer r_3	testTimerPause()	To test pausing countdown timer	timerIsActive() method should return false	PASS
CountDownTimer r_4	testTimerResume()	To test resuming countdown timer	timerIsActive() method should return true	PASS
CountDownTimer r_5	testDrawTimer()	To test whether timer countdown is viewable	Countdown seconds should be visible to player	PASS

CountDownTime r_6	testTimerCompletion()	Simulated player not having action before timer ends	callbackInvoked should be true and timer should be in completed state	PASS
----------------------	-----------------------	--	---	------